المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⴰⵍ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Rapport des travaux pratiques

Architecture Monolithique, Monolithe Modulaire, Architecture Distribuée et Architecture SOA

Filière : Intelligence Artificielle et Cybersécurité

Module :
Architecture Logicielle

Réalisé par :

Abderrahman MACHHOURI

Encadré par :
Pr. BE EL BAGHAZAOUI

Année universitaire : 2025-2026

Table des matières

Liste des figures **2**

Liste des tableaux **3**

1 TP1 : Application Monolithique Simple **4**
1.1 Objectifs 4
1.2 Architecture du Projet 4
1.3 Lien vers Github 5
1.4 Configuration 5
1.4.1 application.properties 5
1.5 Implémentation 5
1.5.1 Entité Product 5
1.5.2 Repository 5
1.5.3 Service 6
1.5.4 Controller 6
1.6 Tests avec Postman 7
1.6.1 Lister tous les produits (GET) 8
1.6.2 Créer un produit (POST) 9
1.6.3 Modifier un produit (PUT) 9
1.6.4 Supprimer un produit (DELETE) 10
1.7 Exercice 10
1.7.1 L'ajout de l'entité Category 10
1.7.2 Test avec Postman 11
1.8 Avantages et Limites du Monolithe 13
1.8.1 Avantages 13
1.8.2 Limites 13
1.9 Conclusion Globale 13

Table des figures

1.1 Architecture en couches du TP1 4
1.2 GET - Lister tous les produits 8
1.3 POST - Créer un produit 9
1.4 PUT - Modifier un produit 9
1.5 DELETE - Supprimer un produit 10
1.6 Relation ManyToOne entre Product et Category 11
1.7 POST - Créer un produit avec sa catégorie 12
1.8 GET - Récupérer un produit avec sa catégorie 12

Liste des tableaux

TP1 : Application Monolithique Simple

1.1 Objectifs

— Créer une application Spring Boot monolithique
— Développer une API REST CRUD complète
— Comprendre l'architecture en couches
— Tester les endpoints avec Postman

1.2 Architecture du Projet

L'application suit une architecture en couches :
— **Entity** : Représentation de la base de données
— **Repository** : Accès aux données
— **Service** : Logique métier
— **Controller** : Exposition des endpoints REST

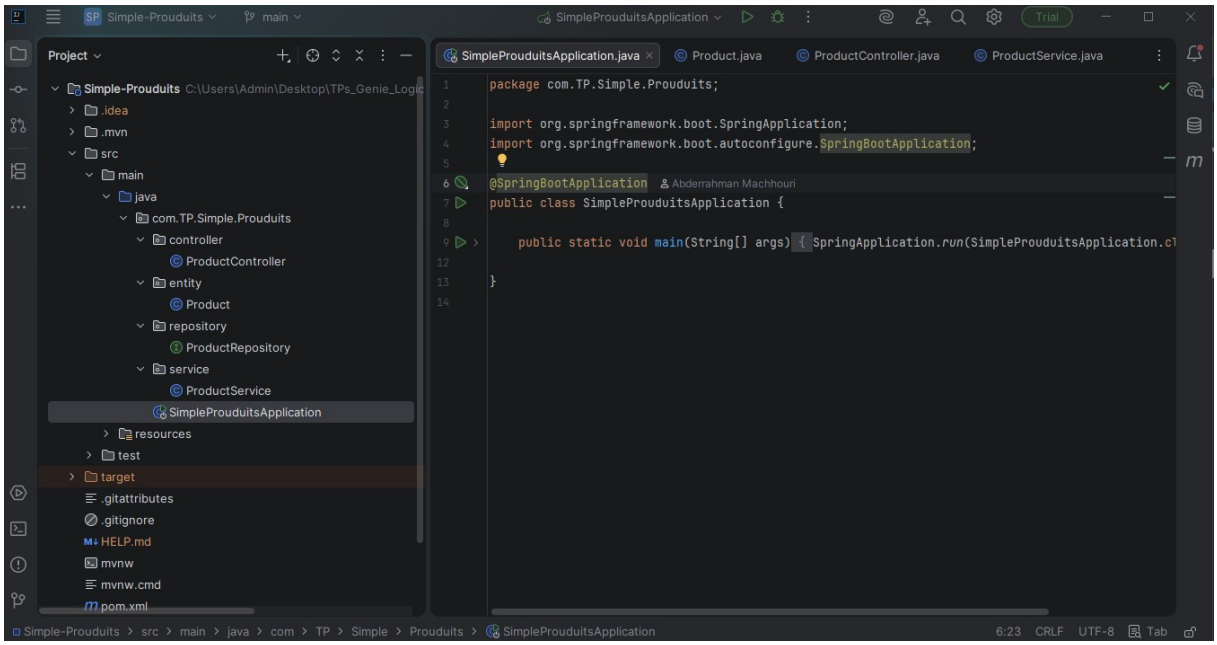

FIGURE 1.1 – Architecture en couches du TP1

1.3 Lien vers Github

Le code source de cette architecture est disponible sur GitHub :
https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/TP1_Monolithe/TP1_Monolithe_Simple
Et pour l'exercice :
https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/TP1_Monolithe/TP1_Monolithe_Simple_Exercice

1.4 Configuration

1.4.1 application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

1.5 Implémentation

1.5.1 Entité Product

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
}
```

1.5.2 Repository

```
public interface ProductRepository
        extends JpaRepository<Product, Long> {

    List<Product> findByNameContainingIgnoreCase(String name);
}
```

1.5.3 Service

Le service contient la logique métier : création, modification, suppression.

```java
package com.TP.Simple.Prouduits.service;
import com.TP.Simple.Prouduits.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import com.TP.Simple.Prouduits.entity.Product;
@Service
@RequiredArgsConstructor
@Transactional
public class ProductService {
    private final ProductRepository ProductRepository;
    @Transactional(readOnly = true)
    public List<Product> getAll() {
        return ProductRepository.findAll();
    }
    @Transactional(readOnly = true)
    public Product getById(Long id) {
        return ProductRepository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException(
                        "Product not found with id: " + id));
    }
    public Product create(Product product) {
        return ProductRepository.save(product);
    }
    public Product update(Long id, Product details) {
        Product product = getById(id);
        product.setName(details.getName());
        product.setDescription(details.getDescription());
        product.setPrice(details.getPrice());
        product.setStock(details.getStock());
        return product;
    }
    public void delete(Long id) {
        Product product = getById(id);
        ProductRepository.delete(product);
    }
}
```

1.5.4 Controller

Les endpoints exposés :

- GET /api/products
- POST /api/products
- PUT /api/products/id
- DELETE /api/products/id

```java
package com.TP.Simple.Prouduits.controller;
```

```java
import com.TP.Simple.Prouduits.entity.Product;
import com.TP.Simple.Prouduits.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import java.util.List;
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
@Validated
public class ProductController {
    private final ProductService ProductService;
    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        List<Product> products = ProductService.getAll();
        return ResponseEntity.ok(products);
    }
    @GetMapping("/{id}")
    public ResponseEntity<Product> getById(@PathVariable Long id) {
        Product product = ProductService.getById(id);
        return ResponseEntity.ok(product);
    }
    @PostMapping
    public ResponseEntity<Product> create(
            @Valid @RequestBody Product product) {
        Product created = ProductService.create(product);
        return ResponseEntity.status(HttpStatus.CREATED).body(
           created);
    }
    @PutMapping("/{id}")
    public ResponseEntity<Product> update(@PathVariable Long id,
                                          @Valid @RequestBody
                                             Product product) {
        Product updated = ProductService.update(id, product);
        return ResponseEntity.ok(updated);
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        ProductService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

1.6 Tests avec Postman

Les tests ont permis de vérifier :

— Création d'un produit

— Modification
— Suppression
— Validation des champs

1.6.1 Lister tous les produits (GET)

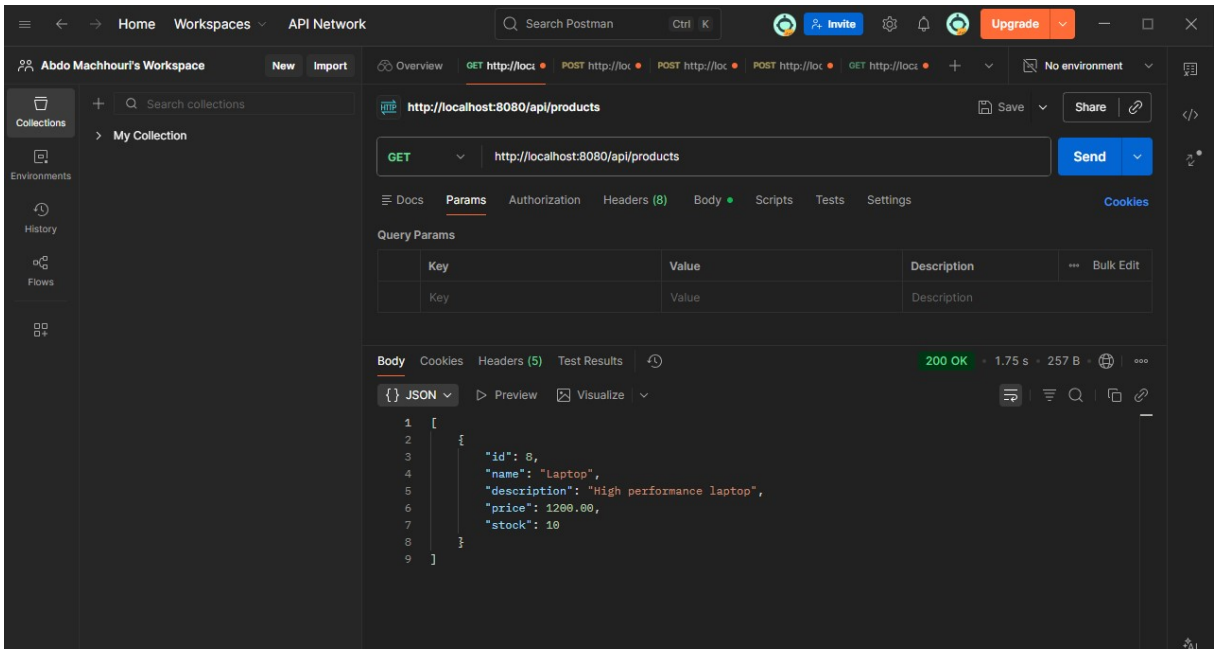

FIGURE 1.2 – GET - Lister tous les produits

1.6.2 Créer un produit (POST)

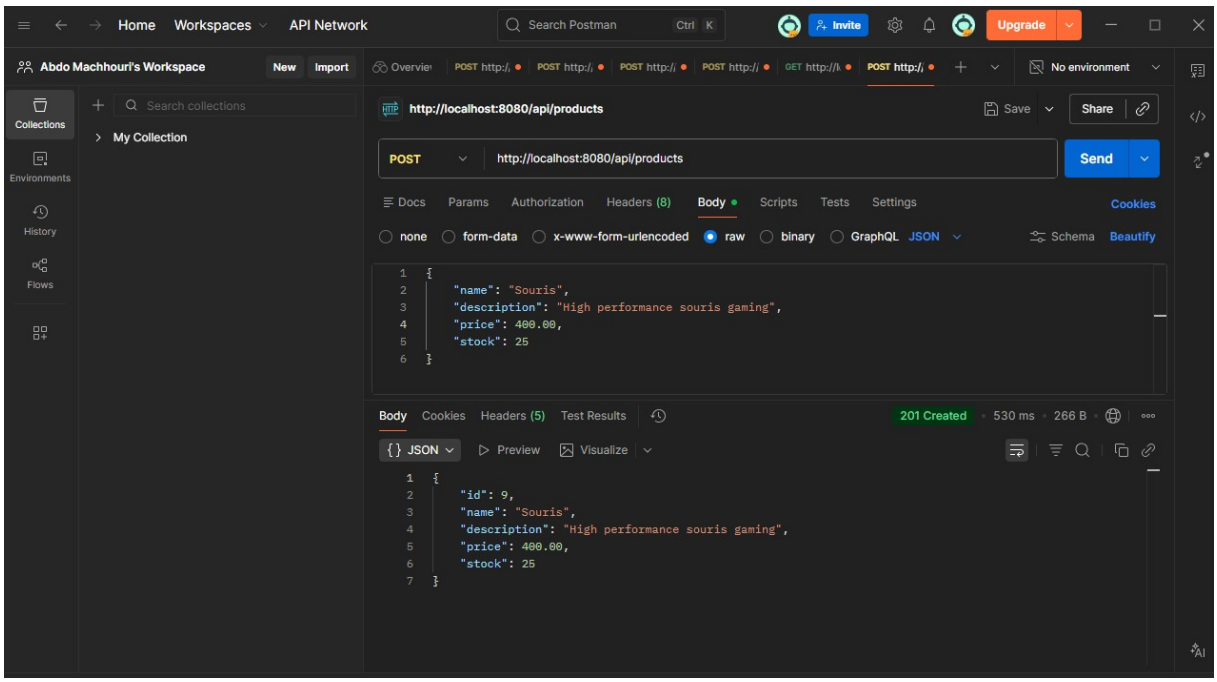

FIGURE 1.3 – POST - Créer un produit

1.6.3 Modifier un produit (PUT)

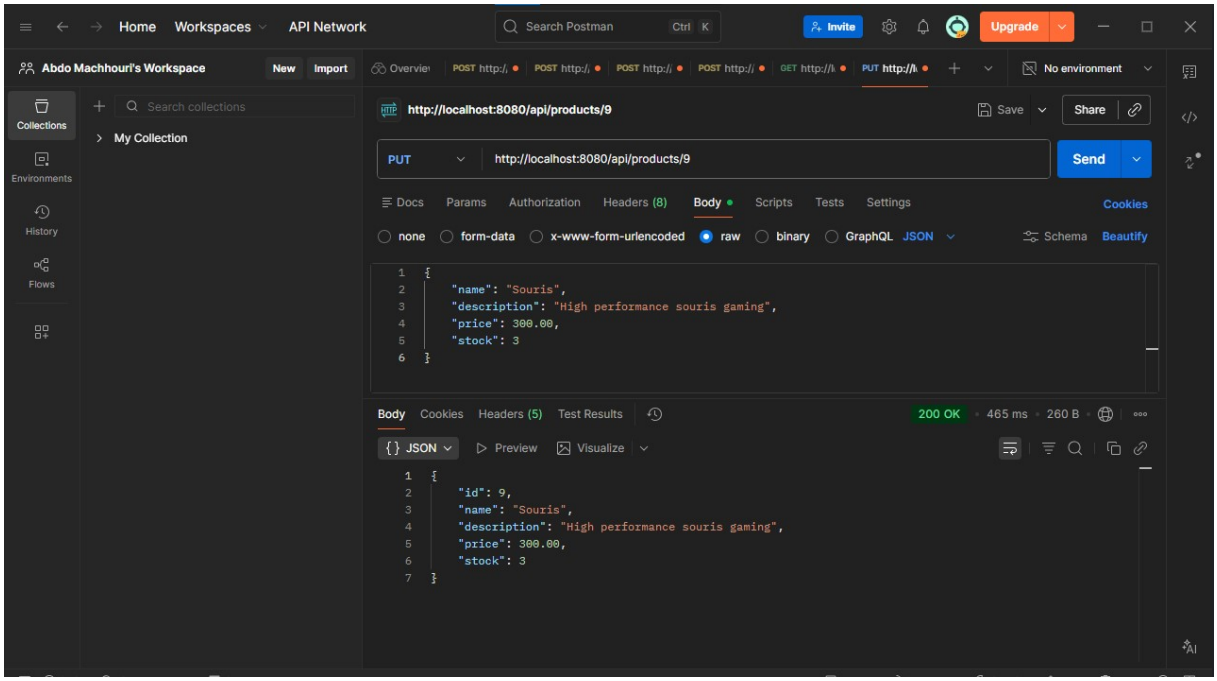

FIGURE 1.4 – PUT - Modifier un produit

1.6.4 Supprimer un produit (DELETE)

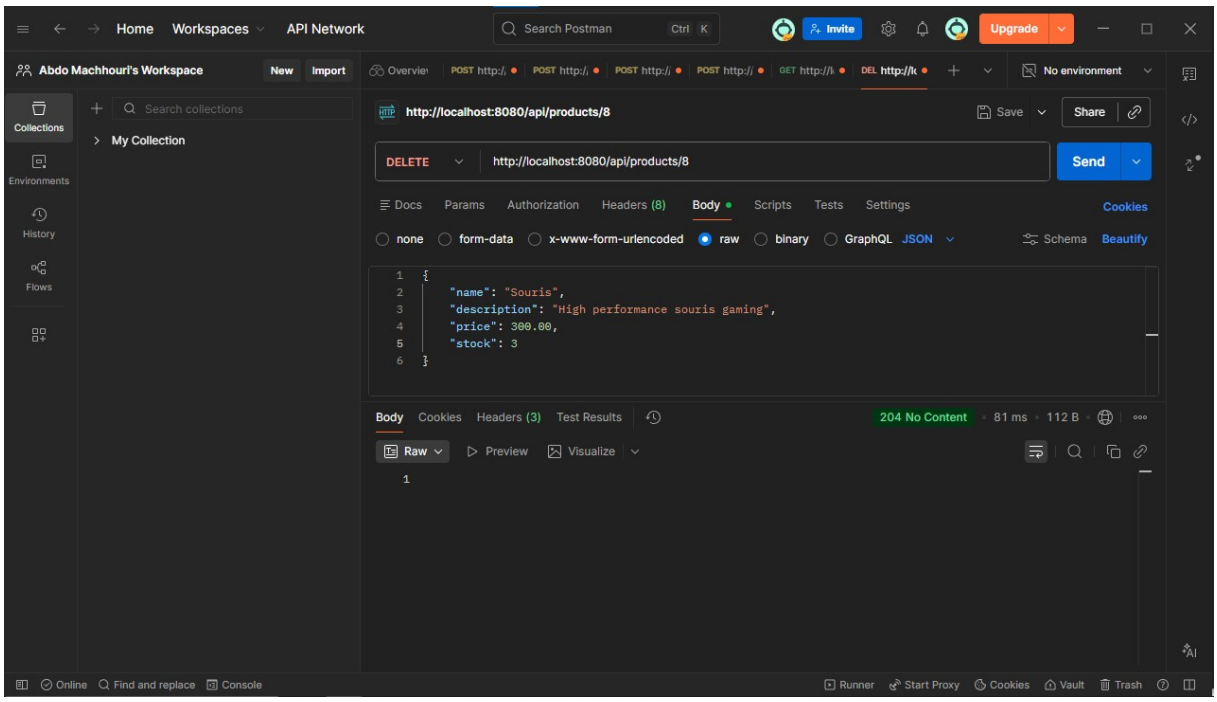

FIGURE 1.5 – DELETE - Supprimer un produit

1.7 Exercice

1.7.1 L'ajout de l'entité Category

On va ajouter l'entité Category et on crée la relation ManyToOne avec l'entité Product.

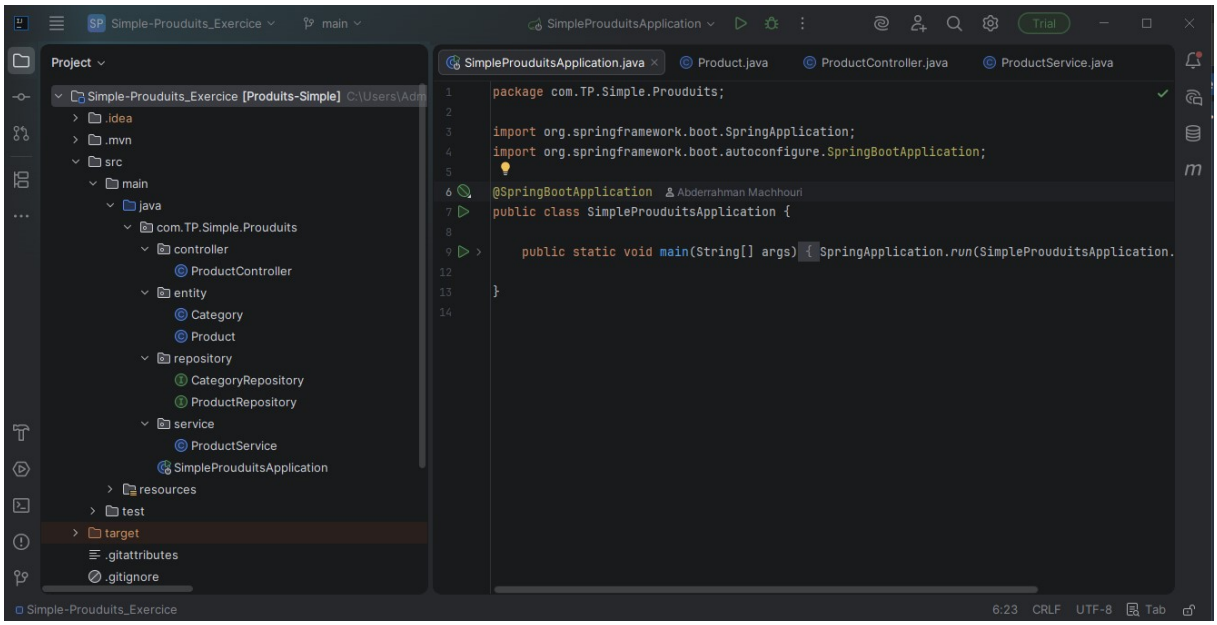

FIGURE 1.6 – Relation ManyToOne entre Product et Category

```
package com.TP.Simple.Prouduits.entity;
import jakarta.persistence.*;
import lombok.*;
@Entity
@Table(name = "category")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name_category;
    @Column(length = 1000)
    private String description;
}
```

1.7.2 Test avec Postman

Dans ce test on va ajouter un produit avec sa catégorie et on va voir si ca marche ou pas. Maintenant on va faire la méthode POST :

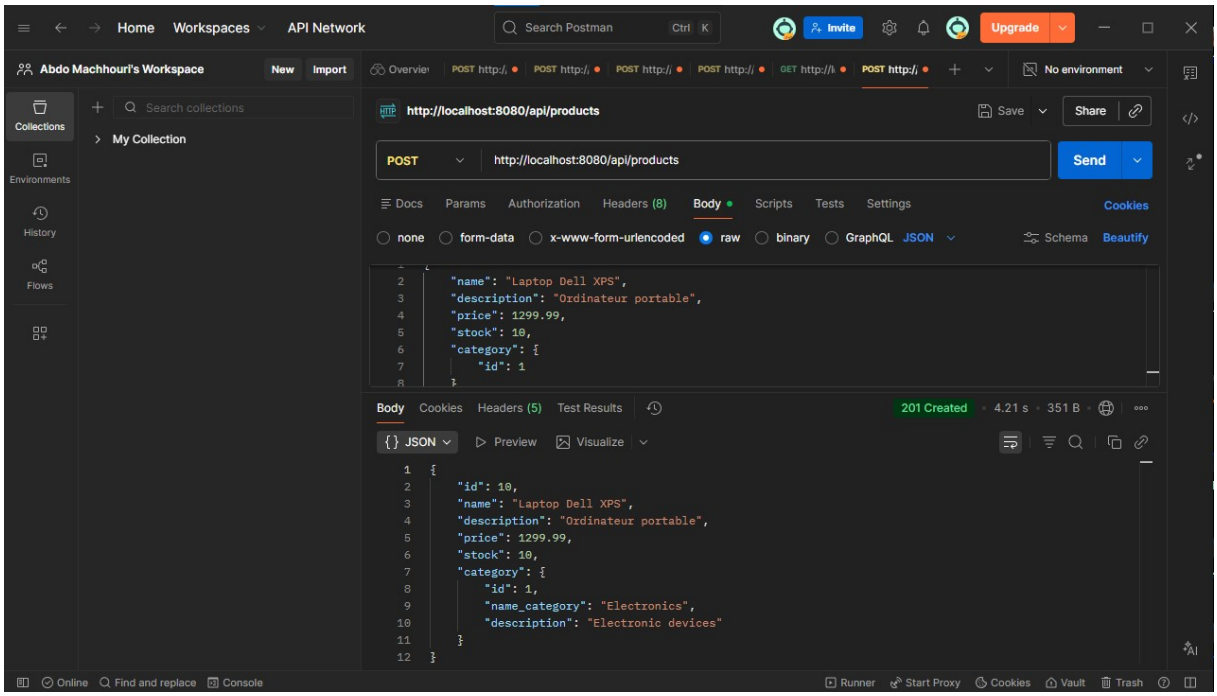

FIGURE 1.7 – POST - Créer un produit avec sa catégorie

Maintenant on va récupérer le produit avec la méthode GET pour voir sa catégorie.

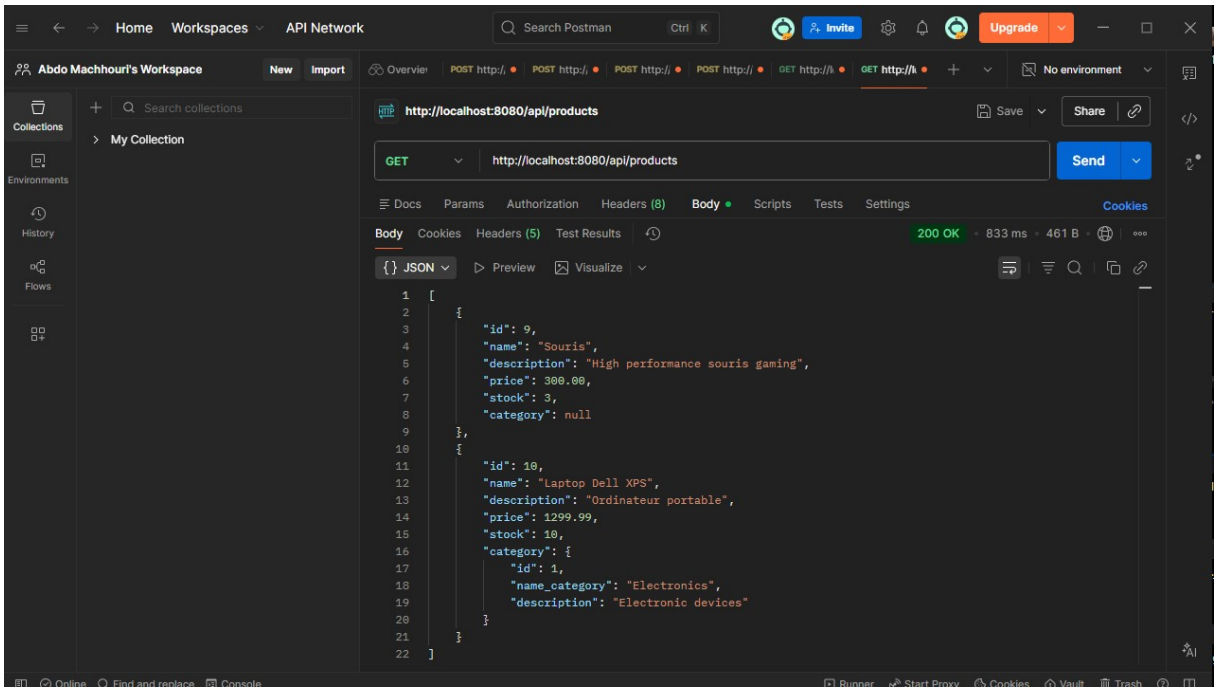

FIGURE 1.8 – GET - Récupérer un produit avec sa catégorie

Et voilà on voit bien la catégorie de notre produit.

1.8 Avantages et Limites du Monolithe

1.8.1 Avantages

- Simple à développer
- Déploiement facile
- Performances élevées

1.8.2 Limites

- Difficulté de maintenance à grande échelle
- Couplage fort entre modules
- Scalabilité limitée

1.9 Conclusion Globale

Ce TP a permis de mettre en place une application monolithique simple avec Spring Boot, en suivant une architecture en couches claire (Entity, Repository, Service, Controller). Nous avons développé une API REST complète pour la gestion de produits, en utilisant à la fois une approche classique et l'option @RepositoryRestResource qui génère automatiquement les endpoints. Les tests avec Postman ont validé le bon fonctionnement des opérations CRUD. L'exercice proposé (ajout d'une entité Category avec une relation ManyToOne vers Product) a illustré l'extension du modèle et l'enrichissement des associations. Ce travail met en évidence la simplicité de développement d'une architecture monolithique, mais aussi ses limites en termes de scalabilité et de maintenabilité à grande échelle, ouvrant la voie vers des architectures distribuées comme celle abordée dans les TP suivants.